САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №5

Выполнил:

Зотеев Максим

БР1.1

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача
Настроить виртуальную машину средствами Ansible (практиковаться рекомендуется на локальной ВМ под управлением VirtualBox/VMWare или других аналогов). Свежая машина приводится в рабочее состояние запуском плейбуков с локальной машины, без ручных действий на самой ВМ.

Ход работы

1. Целевая ВМ и выбор инструмента
Развёртывается микросервисная система сервиса аренды недвижимости из ЛР2–ЛР3: api-gateway, identity-service, property-service, rental-service, messaging-service, PostgreSQL 16 и RabbitMQ 3.13, описанные в общем docker-compose.yml. Вместо локальной ВМ использована арендованная виртуальная машина (VPS) с Ubuntu 24.04 LTS (IP 82.115.222.101): 1 vCPU, 709 МиБ оперативной памяти, диск 8,8 ГБ, изначально доступ только под root по паролю. Для автоматизации выбран Ansible: он не требует агента на сервере (работает по SSH), описывает желаемое состояние декларативно и идемпотентен — повторный запуск без изменений ничего не меняет, а изменения в коде доезжают до сервера тем же самым запуском. Поэтому тот же плейбук запускается и из CI в ДЗ6.

2. Структура Ansible-проекта
Проект лежит в каталоге labs/lab4/: плейбуки написаны при выполнении ЛР4, и на этот же каталог опирается автодеплой из ДЗ6, поэтому в labs/lab5/ он не копируется. Код приложения тоже не дублируется — плейбук берёт его из соседнего labs/lab2/. Задачи разбиты на роли по этапам настройки: common и docker — подготовка сервера, app — развёртывание приложения, nginx — обратный прокси.

```
lab4/
├── ansible.cfg          # inventory, роли, remote_user=deploy, SSH-ключ
├── inventory.ini        # [vps] rental-vps ansible_host=82.115.222.101
├── requirements.yml     # коллекции community.docker, community.general,
│                        #   ansible.posix
├── group_vars/all/
│   ├── vars.yml         # каталоги, список сервисов, swap, порт шлюза
│   └── vault.yml        # секреты, зашифрованы ansible-vault (AES256)
├── bootstrap.yml        # однократная первичная настройка под root
├── deploy.yml           # сервер + приложение + nginx (повторяемый)
└── roles/
    ├── common/          # обновления, пакеты, swap, UFW
    ├── docker/          # Docker Engine и Compose
    ├── app/             # rsync кода, .env, override, сборка, запуск
    └── nginx/           # обратный прокси на api-gateway
```

Секреты (JWT_SECRET, INTERNAL_SERVICE_TOKEN, пароли PostgreSQL и RabbitMQ) сгенерированы случайно (32 символа [A-Za-z0-9]) и хранятся в group_vars/all/vault.yml в зашифрованном виде, поэтому файл можно коммитить. Пароль от vault лежит в локальном .vault_pass, который добавлен в .gitignore. Значения по умолчанию из ЛР2 (change-me-in-prod, rental/rental) на сервере не используются.

3. Первичная настройка и защита SSH (bootstrap.yml)

Единственный ручной шаг — копирование публичного ключа на свежий сервер командой ssh-copy-id root@82.115.222.101 (с вводом пароля root от провайдера). Дальше всё делает Ansible. Плейбук запускается один раз и состоит из трёх play. Первый подключается под root по ключу и создаёт пользователя deploy в группе sudo, кладёт ему публичный ключ и выдаёт sudo без пароля (файл в /etc/sudoers.d проверяется visudo перед записью). Второй play подключается уже под deploy и проверяет, что вход по ключу и sudo работают. Только после этого третий play запрещает вход по паролю и под root: добавляется /etc/ssh/sshd_config.d/00-hardening.conf, конфигурация проверяется командой sshd -t, и SSH перезапускается. Префикс 00- важен: sshd берёт первое встреченное значение параметра, а образ провайдера содержит 50-cloud-init.conf с PasswordAuthentication yes. Такой порядок исключает ситуацию, когда сервер закрывается раньше, чем проверен новый способ входа.

```
PLAY [Создание пользователя deploy] ********************************************
PLAY [Проверка входа под deploy по ключу и работы sudo] ************************
PLAY [Ужесточение SSH] *********************************************************
rental-vps                 : ok=9    changed=5    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
```

Результат: вход под root и по паролю отклоняется сервером, остаётся только вход по ключу под deploy:

```
$ ssh root@82.115.222.101
root@82.115.222.101: Permission denied (publickey).
$ ssh -o PubkeyAuthentication=no deploy@82.115.222.101
deploy@82.115.222.101: Permission denied (publickey).
```

4. Подготовка сервера (роли common и docker)

Роль common обновляет пакеты (apt upgrade safe), ставит rsync, curl, ca-certificates и ufw. Сборка пяти Node.js-образов на 709 МиБ памяти рискует упереться в OOM, поэтому роль приводит swap-файл к 1 ГБ (у провайдера был 768 МБ; размер проверяется через stat, и файл пересоздаётся только при несовпадении), прописывает его в /etc/fstab и выставляет vm.swappiness=10. Межсетевой экран UFW настраивается так: входящие по умолчанию запрещены, разрешены только 22/tcp и 80/tcp, причём правило для SSH добавляется до включения UFW, чтобы не потерять текущее подключение. Также роль добавляет в authorized_keys пользователя deploy отдельный ключ GitHub Actions для автодеплоя из ДЗ6, не заменяя личный ключ из bootstrap.yml. Роль docker подключает официальный apt-репозиторий Docker (ключ в /etc/apt/keyrings/docker.asc, источник в формате deb822) и ставит docker-ce, docker-ce-cli, containerd.io, docker-buildx-plugin и docker-compose-plugin. Docker включается в автозапуск, пользователь deploy добавляется в группу docker, после чего соединение переоткрывается (meta: reset_connection), чтобы группа применилась. После установки всех пакетов и до сборки образов удаляются скачанные .deb-файлы: на 8,8 ГБ диска они занимали больше гигабайта.

5. Развёртывание приложения (роль app)

Код lab2 копируется в /opt/rental модулем ansible.posix.synchronize (rsync): передаются только изменённые файлы, удалённые локально файлы удаляются и на сервере. Исключаются node_modules, dist, локальный .env, отчёты и коллекция Postman, а также серверные файлы (.env, docker-compose.prod.yml, метка сборки) — исключённые файлы rsync не удаляет и при --delete. Файлы сравниваются по содержимому (checksum: true), а время изменения, владелец и группа не переносятся (times: false, owner: false, group: false): при сравнении по времени каждый запуск из свежего клона в CI считался бы изменением всего кода и запускал бы полную пересборку.

Секреты попадают на сервер двумя шаблонами. Файл .env (права 0600) задаёт JWT_SECRET, JWT_EXPIRES_IN и INTERNAL_SERVICE_TOKEN, которые docker-compose.yml из lab2 уже читает из окружения. Пароли БД и брокера в lab2 записаны прямо в compose-файле, поэтому они переопределяются в серверном docker-compose.prod.yml, который подключается вторым файлом (-f docker-compose.yml -f docker-compose.prod.yml). Пароль PostgreSQL применяется только при первой инициализации тома pgdata — на свежем сервере это так, а при смене пароля в vault том придётся пересоздать.

Тот же override закрывает порты. Опубликованные Docker порты обходят UFW: Docker сам добавляет правила iptables, поэтому 3001–3004, 5432, 5672, 15672 и 8080 из docker-compose.yml были бы доступны из интернета, несмотря на межсетевой экран. В override у всех сервисов публикация портов сбрасывается тегом !reset, а у api-gateway заменяется тегом !override на 127.0.0.1:8080 — к нему обращается только nginx на том же сервере. Обычное слияние compose дописало бы порты к спискам, а не заменило их.

```
services:
  postgres:
    ports: !reset []
    environment:
      POSTGRES_PASSWORD: "<из vault>"
  rabbitmq:
    ports: !reset []
    environment:
      RABBITMQ_DEFAULT_PASS: "<из vault>"
  identity-service:          # аналогично property-, rental-, messaging-service
    ports: !reset []
    environment:
      DB_PASSWORD: "<из vault>"
      RABBITMQ_URL: "amqp://rental:<из vault>@rabbitmq:5672"
  api-gateway:
    ports: !override
      - "127.0.0.1:8080:8080"
```

Пересобираются только затронутые сервисы. Из вывода rsync берётся список изменённых файлов (строки, в которых изменились только атрибуты, отбрасываются): изменения в services/<сервис>/ требуют пересборки этого сервиса, а изменения в packages/shared и корневых package*.json — всех сервисов, кроме api-gateway, который собирается из своего каталога. Список записывается в /opt/rental/.pending-build сразу после копирования кода и удаляется только после успешной сборки, поэтому прерванный запуск не теряет пересборку — следующий её доделает. При самом первом

развёртывании собираются все образы. Сборка идёт по одному сервису (docker compose build <сервис>), а не параллельно: на одном vCPU и 709 МиБ параллельная установка npm-зависимостей пяти сервисов не помещается в память. Затем модуль community.docker.docker_compose_v2 поднимает стек без пересборки и пересоздаёт только контейнеры с изменившимися образами или настройками, плейбук ждёт ответа http://127.0.0.1:8080/health, после чего удаляются старые образы и кэш сборки: на 8,8 ГБ диска старые и новые образы вместе с кэшем сборки не помещаются (при попытке сохранить кэш диск заполнился до 100%).
При первом развёртывании выяснилось, что package-lock.json в lab2 ссылались на внутреннее зеркало npm.yandex-team.ru, недоступное с VPS: npm install внутри сборки зависал и падал, а следом сборка останавливалась на «tsc: not found». Ссылки во всех шести lock-файлах заменены на https://registry.npmjs.org/ — хеши integrity при этом остаются верными, так как пакеты побайтно те же; все 261 уникальный адрес проверены. Это исправление делает lab2 собираемой вне корпоративной сети.

6. nginx как обратный прокси (роль nginx)
nginx ставится из репозитория Ubuntu, сайт по умолчанию отключается, вместо него включается сайт rental, который слушает 80 порт и проксирует все запросы на api-gateway. Шлюз получает исходные Host и адрес клиента через заголовки. При изменении конфигурации срабатывают два обработчика по порядку: nginx -t и, только если конфигурация валидна, reload. Обработчики применяются сразу после роли nginx (meta: flush_handlers), а не в конце плейбука: иначе при ошибке на этапе сборки приложения перезагрузка nginx потерялась бы, и следующий запуск её уже не вызвал бы.

```
upstream rental_gateway {
    server 127.0.0.1:8080;
}

server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name 82.115.222.101 _;
    client_max_body_size 10m;

    location / {
        proxy_pass http://rental_gateway;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }
}
```

Последний шаг deploy.yml с локальной машины запрашивает http://82.115.222.101/health и падает, если приложение снаружи не отвечает.

7. Проверка

Логи этого раздела сняты при первоначальной настройке ВМ. После добавления задачи с ключом GitHub Actions повторный запуск без изменений даёт ok=33 вместо ok=32, по-прежнему с changed=0 (см. отчёт ДЗ6). Прогон deploy.yml с установкой nginx и сборкой всех образов, а также самые долгие задачи первой сборки (1 vCPU):

```
rental-vps                 : ok=37   changed=9    unreachable=0    failed=0
skipped=5    rescued=0    ignored=0

app : Сборка образа сервиса ---------------------------------------- 343.80s
app : Запуск контейнеров ------------------------------------------- 68.66s
app : Очистка старых образов и кэша сборки ------------------------- 29.86s
```

Повторный запуск без изменений в коде занимает около минуты и ничего не меняет — контейнеры не пересоздаются:

```
rental-vps                 : ok=32   changed=0    unreachable=0    failed=0
skipped=10   rescued=0    ignored=0
```

После изменения файла в lab2 (services/property-service/src/index.ts) тот же запуск копирует изменения, пересобирает только образ property-service и пересоздаёт только его контейнер, остальные шесть контейнеров продолжают работать; следующий запуск снова даёт changed=0:

```
rental-vps                 : ok=36   changed=3    unreachable=0    failed=0
skipped=6    rescued=0    ignored=0
rental-vps                 : ok=32   changed=0    unreachable=0    failed=0
skipped=10   rescued=0    ignored=0
```

Приложение доступно извне через nginx:

```
$ curl -i http://82.115.222.101/health
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Fri, 16 Oct 2026 21:06:20 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 39
Connection: keep-alive
X-Powered-By: Express
Access-Control-Allow-Origin: *
ETag: W/"27-OEeR3MgSKG+WBNBm2ZN8Bz3Z0Ag"

{"status":"ok","service":"api-gateway"}

$ curl -s http://82.115.222.101/api/v1/property-types
[{"id":1,"name":"Квартира"},{"id":2,"name":"Дом"},{"id":3,"name":"Комната"},
{"id":4,"name":"Апартаменты"},{"id":5,"name":"Студия"}]
```

Снаружи слушают только SSH и nginx, шлюз привязан к 127.0.0.1, внутренние сервисы, PostgreSQL и RabbitMQ не публикуются вовсе:

```
$ ssh deploy@82.115.222.101 "sudo ss -tlnp"
State Local Peer
LISTEN 0.0.0.0:22 sshd
LISTEN 0.0.0.0:80 nginx
LISTEN 127.0.0.1:8080 docker-proxy
LISTEN [::]:22 sshd
LISTEN [::]:80 nginx

$ ssh deploy@82.115.222.101 "sudo ufw status"
Status: active

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW       Anywhere
80/tcp                     ALLOW       Anywhere
22/tcp (v6)                ALLOW       Anywhere (v6)
80/tcp (v6)                ALLOW       Anywhere (v6)
```

```
$ docker ps (на сервере)
NAMES              STATUS                       PORTS
rental_gateway     Up 51 seconds                127.0.0.1:8080->8080/tcp
rental_messaging   Up 14 seconds                3004/tcp
rental_rental      Up 14 seconds                3003/tcp
rental_property    Up 14 seconds                3002/tcp
rental_identity    Up 14 seconds                3001/tcp
rental_rabbitmq    Up 51 seconds (healthy)      4369/tcp, 5671-5672/tcp,
15671-15672/tcp, 15691-15692/tcp, 25672/tcp
rental_postgres    Up 51 seconds (healthy)      5432/tcp
```

Работа всей системы через публичный адрес подтверждена прогоном Postman-коллекции из ЛР2 через newman с base_url=http://82.115.222.101/api/v1:

	executed	failed
iterations	1	0
requests	62	0
test-scripts	62	0
prerequest-scripts	64	0
assertions	127	0
total run duration: 10.7s		
total data received: 23.55kB (approx)		
average response time: 130ms [min: 63ms, max: 994ms, s.d.: 130ms]		

Сервер перезагружен для проверки автозапуска: через ~30 секунд вернулись SSH, UFW, swap 1 ГБ, Docker, nginx и все семь контейнеров, /health отвечает. Docker после перезагрузки запускает контейнеры одновременно, без учёта depends_on, и RabbitMQ на одном vCPU стартует дольше, чем длятся 10 попыток подключения сервисов по 2 секунды, поэтому каждый прикладной сервис один раз перезапускается политикой restart: unless-stopped, после чего подключается к брокеру.

Заключение

В рамках ЛР5 виртуальная машина с Ubuntu 24.04 настроена полностью средствами Ansible, и на ней развёрнуто микросервисное приложение из ЛР2–ЛР3. Однократный bootstrap.yml создаёт пользователя deploy и закрывает вход по паролю и под root, предварительно проверив вход по ключу. Повторяемый deploy.yml подготавливает сервер (обновления, swap, UFW, Docker), копирует код по rsync, подставляет секреты из Ansible Vault, собирает и запускает контейнеры и настраивает nginx как обратный прокси на api-gateway. Снаружи доступны только порты 22 и 80, внутренние порты закрыты на уровне compose, так как Docker обходит UFW. Повторный запуск без изменений даёт changed=0, изменения кода доставляются тем же запуском, а приложение переживает перезагрузку сервера. Работоспособность подтверждена прогоном Postman-коллекции через публичный адрес: 62 запроса, 127 проверок, ошибок нет. Этот же плейбук используется для автоматического развёртывания в ДЗ6.